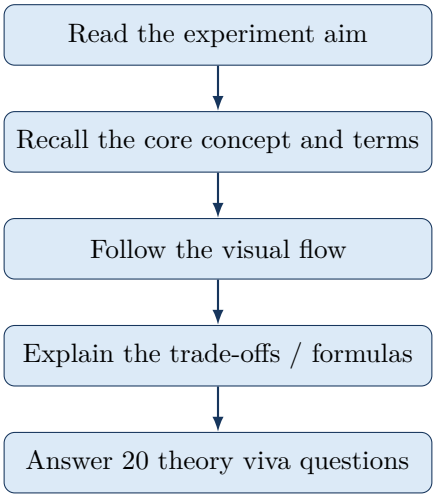
LP-III Theory Revision Guide

DAA Experiments 1–4 and ML Experiments 1–5

Point-wise notes • memory flow diagrams • 180 theory viva questions with answers plus expanded conceptual notes



How to use this guide

- Learn the bold terms and flow diagrams first.
- Give viva answers in short, numbered points rather than long paragraphs.
- Memorize the key comparisons, formulas, assumptions, and limitations.
- Dataset columns and exact outputs can vary by supplied file; do not memorize invented results.

Prepared as theory-only revision material. It intentionally omits program listings and line-by-line code explanations.

Contents

1 DAA Experiment 1: Fibonacci Using Iteration and Recursion 2

2 DAA Experiment 2: Huffman Encoding Using a Greedy Strategy 3

3 DAA Experiment 3: Fractional Knapsack Using Greedy Method 5

4 DAA Experiment 4: 0–1 Knapsack Using Dynamic Programming 7

5 ML Foundations: Common Theory 9

6 ML Experiment 1: Uber Fare Prediction with Linear Regression and Random Forest 9

7 ML Experiment 2: Email Spam Detection Using KNN and SVM 11

8 ML Experiment 3: Bank Customer Churn Prediction Using an ANN 13

9 ML Experiment 4: Diabetes Prediction Using KNN 15

10 ML Experiment 5: Sales Customer Segmentation Using K-Means / Hierarchical Clustering 17

11 Cross-experiment memory map 19

1 DAA Experiment 1: Fibonacci Using Iteration and Recursion

Aim and theory to know

- **Fibonacci sequence**: each term is the sum of the previous two terms; commonly $F_0 = 0$, $F_1 = 1$, and $F_n = F_{n-1} + F_{n-2}$.
- **Recursion**: a function calls itself to solve a smaller instance.
- **Base case**: stops recursive calls; without it, recursion may continue until failure.
- **Recursive approach**: directly follows the recurrence; simple to express but repeats many calculations in the naive version.
- **Iterative approach**: uses a loop and stores the previous two values; avoids recursive call overhead.
- **Time complexity**: naive recursion is exponential, commonly described as $O(2^n)$; iteration is $O(n)$.
- **Space complexity**: naive recursion uses $O(n)$ call-stack space; an iterative two-variable version uses $O(1)$ auxiliary space.
- **Complexity analysis**: counts how resource use grows with input size, not exact seconds on one computer.



Deeper theory: recurrence, growth, and trade-offs

- **Mathematical definition**: $F_0 = 0$, $F_1 = 1$; for $n \geq 2$, $F_n = F_{n-1} + F_{n-2}$. State the indexing convention before answering examples.
- **Call-tree reasoning**: naive recursion branches into calls for $n-1$ and $n-2$. Many branches request the same values, such as F_3 being recomputed while evaluating F_5 and F_4.
- **Recurrence for running time**: $T(n) = T(n-1) + T(n-2) + \Theta(1)$. Its growth is exponential, more precisely $\Theta(\varphi^n)$ where $\varphi = (1+\sqrt{5})/2$; $O(2^n)$ is a simpler loose upper bound.
- **Iterative invariant**: after each loop iteration, the two variables hold consecutive Fibonacci values. Updating the pair advances the invariant by one index.
- **Three implementation families**: naive recursion recomputes; memoization caches each index; tabulation fills values from small to large. Memoization/tabulation take $O(n)$ time; storing the full table costs $O(n)$ space, while a rolling pair uses $O(1)$ auxiliary space.
- **Input and arithmetic caveat**: runtime analysis assumes arithmetic operations take constant time. Very large Fibonacci values contain many digits, so big-integer arithmetic costs more than a constant-time operation.
- **Method selection**: recursion is pedagogically direct; iteration is usually best for one value when low memory and predictable performance matter; memoization is convenient when only some subproblems are needed.
- **Complexity vocabulary**: best/average/worst case describe input behavior; O is an upper bound, Ω a lower bound, and Θ a tight asymptotic bound.

Trace example: $F_0 = 0, F_1 = 1$; update $(a, b) \leftarrow (b, a+b)$ gives $(0,1) \to (1,1) \to (1,2) \to (2,3) \to (3,5)$.

Theory viva: 20 questions and answers

1. **What is the Fibonacci sequence?** Each term after the first two is the sum of the two preceding terms.
2. **State the recurrence relation.** $F_n = F_{n-1} + F_{n-2}$, with base values such as $F_0 = 0$ and $F_1 = 1$.

3. **What is recursion?** A technique in which a function calls itself on smaller inputs.
4. **What is a base case?** A stopping condition that returns a known result without further recursive calls.
5. **Why is a base case necessary?** It prevents unbounded recursion and makes the recurrence terminate.
6. **What is iteration?** Repeating a block of operations using a loop until a condition is met.
7. **Why is naive recursive Fibonacci slow?** It recalculates the same Fibonacci values many times.
8. **What is the time complexity of naive recursion?** Exponential; a common upper-bound description is $O(2^n)$.
9. **What is the time complexity of the iterative method?** $O(n)$, because the loop advances through the sequence once.
10. **What is the auxiliary space of an iterative two-variable method?** $O(1)$.
11. **What is the stack in recursion?** Memory that stores active function calls and their local state.
12. **What is stack overflow?** Failure caused by exhausting the available call-stack space, often from excessive recursion.
13. **Can recursion be easier to understand than iteration?** Yes; it mirrors the mathematical definition, though it may use more resources.
14. **What is a repeated subproblem?** A smaller calculation that is solved more than once.
15. **How can repeated Fibonacci work be avoided?** Memoization or tabulation stores previously computed values.
16. **What is dynamic programming?** A method that stores solutions to overlapping subproblems to avoid recomputation.
17. **What is input size here?** Usually the index n whose Fibonacci number is requested.
18. **Does Big-O give exact runtime?** No; it describes asymptotic growth and suppresses constants and lower-order terms.
19. **Why compare recursive and iterative approaches?** To understand readability, runtime growth, and memory overhead.
20. **What should be checked for boundary inputs?** Values such as $n = 0$, $n = 1$, negative input handling, and the sequence convention.

2 DAA Experiment 2: Huffman Encoding Using a Greedy Strategy

Aim and theory to know

- **Huffman coding**: lossless compression using variable-length binary codes.
- **Frequency**: number of occurrences of each symbol; frequent symbols generally receive shorter codes.
- **Greedy choice**: repeatedly combine the two lowest-frequency nodes.
- **Min-priority queue**: efficiently retrieves the two minimum-weight nodes.
- **Binary tree**: leaves represent symbols; internal nodes represent combined frequencies.
- **Prefix-free code**: no symbol code is the prefix of another; decoding is unambiguous.
- **Encoding**: traverse the tree, commonly assigning 0 to one branch and 1 to the other.

- **Decoding**: follow bits through the tree until a leaf is reached, emit its symbol, then restart at the root.
- **Complexity**: with a priority queue, building the tree is commonly $O(m \log m)$ for m distinct symbols.
- **Limitation**: compression depends on symbol frequencies; code construction requires the coding tree or codebook to decode.



Deeper theory: construction, optimality, and decoding

- **Coding objective**: minimize the weighted external path length $\sum_i f_i \ell_i$, where f_i is symbol frequency and ℓ_i is its code length. If frequencies are probabilities, expected code length is $\sum_i p_i \ell_i$ bits/symbol.
- **Why merging works**: the two least-frequent symbols can be placed as deepest sibling leaves in an optimal prefix tree; merging them into a pseudo-symbol reduces the problem to a smaller instance. This is the key greedy-choice argument.
- **Construction trace**: with frequencies A:5, B:9, C:12, D:13, E:16, F:45, repeatedly merge the two minimum weights. Ties can produce different trees/codes but the same optimal cost.
- **Tree representation**: each leaf stores a symbol and frequency; each internal node stores the sum of child frequencies. A code is the root-to-leaf path, using a consistent 0/1 convention.
- **Encoding/decoding**: encoding replaces each symbol with its codeword. Decoding consumes bits from the root until a leaf is reached, emits the symbol, and restarts at the root.
- **Prefix-free property**: no codeword is the prefix of another, so the bitstream can be decoded without separators. Huffman codes are instantaneous and uniquely decodable.
- **Complexity**: for m distinct symbols, a min-heap gives $O(m \log m)$ construction; assigning codes by tree traversal is $O(m)$, excluding the total output-bit length.
- **Compression ratio**: compare original representation size with encoded size, including the cost of transmitting/storing the tree or codebook. Tiny inputs may not shrink after metadata overhead.
- **Limitations and edge cases**: one-symbol input may require a convention such as code 0; equal frequencies may yield different valid codes; static Huffman coding needs frequency knowledge before encoding, while adaptive variants update statistics.
- **Huffman versus fixed length**: fixed-length codes use $\lceil \log_2 m \rceil$ bits per symbol; Huffman exploits unequal frequencies but code lengths remain integer numbers of bits and may not be globally optimal for every block-coding objective.

Theory viva: 20 questions and answers

1. **What is Huffman coding?** A lossless compression algorithm that creates variable-length, prefix-free codes based on symbol frequencies.
2. **Why is it called a greedy algorithm?** It repeatedly makes the locally optimal choice of combining the two least-frequent nodes.
3. **What is a symbol frequency?** The count of how often a symbol appears in the input.
4. **Which symbols usually get shorter codes?** More frequent symbols.
5. **What is a prefix-free code?** A code in which no complete symbol code is the prefix of another code.
6. **Why is prefix-free coding useful?** It allows unambiguous decoding without explicit separators between symbols.

7. **What is a Huffman tree?** A binary tree whose leaves represent symbols and whose path bits form their codes.
8. **What is stored in an internal node?** Typically the sum of the frequencies of its two children.
9. **What is the greedy step?** Remove and merge the two nodes with the smallest weights.
10. **Why use a min-priority queue?** It efficiently finds the two smallest-frequency nodes at each step.
11. **What does a leaf represent?** An original symbol.
12. **What do 0 and 1 mean in a code?** They represent the chosen left/right branches; the exact assignment is arbitrary if consistent.
13. **How is a symbol encoded?** By the bit sequence on the path from root to its leaf.
14. **How is a bitstream decoded?** Follow its bits from the root until reaching a leaf, output that symbol, then restart.
15. **Is Huffman coding lossy?** No; it preserves the original symbols exactly.
16. **What is the usual tree-building complexity?** $O(m \log m)$ using a priority queue for m distinct symbols.
17. **Does Huffman always produce equal-length codes?** No; code lengths vary according to frequencies and tree structure.
18. **What happens if all symbol frequencies are equal?** The resulting tree still gives a valid prefix code; code lengths depend on the number of symbols and tie choices.
19. **What information is needed to decode?** The code tree or an equivalent codebook, along with the encoded bits.
20. **What is a limitation of Huffman coding?** It compresses according to symbol statistics and may provide little benefit when frequencies are nearly uniform or overhead is significant.

3 DAA Experiment 3: Fractional Knapsack Using Greedy Method

Aim and theory to know

- **Problem**: select items with values and weights without exceeding capacity W.
- **Fractional rule**: any fraction of an item may be selected.
- **Value density**: $r_i = v_i/w_i$; value obtained per unit weight.
- **Greedy strategy**: sort items by decreasing value density and take the most valuable density first.
- **Capacity check**: take a whole item if it fits; otherwise take the fraction that fills the remaining capacity.
- **Why it works**: for divisible items, replacing lower-density weight with higher-density weight cannot reduce total value.
- **Complexity**: sorting dominates, usually $O(n \log n)$; selection after sorting is $O(n)$.
- **Contrast**: this greedy strategy is optimal for fractional knapsack, but not generally for 0–1 knapsack.



Deeper theory: why the greedy choice is valid

- **Problem model**: capacity W; item i has value v_i, weight w_i, and decision variable x_i. Maximize $\sum_i v_i x_i$ subject to $\sum_i w_i x_i \le W$.
- **Fractional constraint**: $0 \le x_i \le 1$. Taking a fraction of an item is allowed, unlike the 0–1 variant.
- **Value density**: $r_i = v_i/w_i$ for positive weights. Sort items in non-increasing density; take each whole while possible, then take the fraction that fills remaining capacity.
- **Exchange argument**: if a solution takes a lower-density unit while omitting an available higher-density unit, swapping weight from the lower-density item to the higher-density one cannot reduce value. Repeating this exchange justifies the greedy order.
- **Worked symbolic trace**: if capacity remaining is R and the next item weighs $w_i > R$, take fraction $x_i = R/w_i$ and gain value $x_i v_i$; capacity becomes zero and the process ends.
- **Complexity**: sorting costs $O(n \log n)$; the scan costs $O(n)$; total is $O(n \log n)$. If items are already sorted, the scan is linear.
- **Assumptions**: item values and weights are known; weights are positive; divisible quantities are meaningful; capacity is a single constraint.
- **Why not use the same greedy rule for 0–1 knapsack?**: indivisibility prevents the exchange step from taking an arbitrary fraction. Highest value density can block a better combination of whole items.
- **Edge cases**: zero-weight items require explicit handling to avoid division by zero; negative values should not be selected when selection is optional; capacity 0 yields value 0 for positive-weight items.
- **Optimality distinction**: density greedy is optimal for fractional knapsack, but not in general for 0–1 knapsack.

Theory viva: 20 questions and answers

1. **What is the fractional knapsack problem?** Maximize total value under a weight limit when item fractions are allowed.
2. **What is the greedy criterion?** Value-to-weight ratio, v_i/w_i.
3. **Why sort by ratio?** It maximizes value gained per unit of capacity at each step.
4. **Can an item be divided?** Yes, any fraction from 0 to 1 may be selected.
5. **When is an item taken completely?** When its full weight fits in the remaining capacity.
6. **When is only a fraction taken?** When the next item exceeds remaining capacity; take the fraction that fills it.
7. **What is the objective?** Maximize total value without exceeding capacity.
8. **What does capacity mean?** The maximum total weight allowed.
9. **What is value density?** Value per unit weight.
10. **What is the time complexity?** Usually $O(n \log n)$ due to sorting.
11. **What is the complexity after sorting?** $O(n)$ to scan the items.
12. **Why is the greedy strategy optimal here?** Items are divisible, so higher value density can replace lower density without loss.
13. **Does the same greedy rule always solve 0–1 knapsack?** No, because items cannot be split in 0–1 knapsack.
14. **What if capacity is zero?** No positive-weight item can be selected; total value is zero.

15. **What if an item has zero weight?** It needs special handling; a positive-value zero-weight item can be included without consuming capacity.
16. **What is a feasible solution?** A selection whose total weight does not exceed capacity.
17. **What is an optimal solution?** A feasible solution with maximum total value.
18. **Why must weights be positive for the ratio rule?** The ratio v_i/w_i is undefined at zero weight and behaves differently for invalid weights.
19. **What is the key memory phrase?** "Ratio first, fill capacity, then fraction."
20. **Give one application.** Allocating divisible resources such as cargo space or bandwidth when value per unit is known.

4 DAA Experiment 4: 0–1 Knapsack Using Dynamic Programming

Aim and theory to know

- **Problem**: choose whole items to maximize value while total weight stays within capacity.
- **0–1 constraint**: each item is either selected (1) or not selected (0); fractions are not allowed.
- **Dynamic programming**: store answers to smaller subproblems.
- **State**: $DP[i][w]$ is the maximum value using the first i items with capacity w.
- **Base cases**: $DP[0][w] = 0$ and $DP[i][0] = 0$.
- **Transition**: if $w_i > w$, copy $DP[i-1][w]$; otherwise take the maximum of excluding or including item i.
- **Optimal substructure**: an optimal solution is composed from optimal solutions to smaller states.
- **Complexity**: table-based DP uses $O(nW)$ time and $O(nW)$ space; this is pseudo-polynomial because it depends on numeric capacity W.
- **Contrast with fractional knapsack**: fractional permits portions and has a ratio-based greedy optimum; 0–1 requires DP or other exact/approximate methods.

$$DP[i][w] = \begin{cases} DP[i-1][w], & w_i > w, \\ \max(DP[i-1][w],\, v_i + DP[i-1][w-w_i]), & w_i \le w. \end{cases}$$

Final answer is $DP[n][W]$; trace table to recover items

Deeper theory: state design and dynamic programming

- **Decision variables**: $x_i \in \{0, 1\}$; each item is either taken whole or not taken. Objective and capacity constraint are the same as knapsack, but fractions are forbidden.
- **DP state**: $DP[i][w]$ is the maximum value obtainable using the first i items with capacity at most w.
- **Transition**: if $w_i > w$, item i cannot fit, so $DP[i][w] = DP[i-1][w]$. Otherwise $DP[i][w] = \max(DP[i-1][w], v_i + DP[i-1][w-w_i])$.
- **Base cases**: $DP[0][w] = 0$ for all capacities and $DP[i][0] = 0$ for all item counts.
- **Why DP applies**: the problem has optimal substructure and overlapping subproblems. The optimal decision for item i depends on smaller states using earlier items.
- **Complexity**: table DP takes $O(nW)$ time and typically $O(nW)$ space. This is pseudo-polynomial: it depends on numeric capacity W, not only the number of bits needed to encode W.

- **Space optimization**: a one-dimensional array can reduce memory to $O(W)$, but capacities must be processed in descending order. Ascending order can reuse the same item multiple times, accidentally changing the problem to unbounded knapsack.
- **Solution reconstruction**: store decisions or backtrack from $DP[n][W]$; if the current state differs from the row above, item i was selected and capacity is reduced by w_i.
- **Branch and bound alternative**: explore include/exclude decisions in a state-space tree; an optimistic upper bound prunes branches that cannot beat the best solution found. It can be faster on some inputs but worst-case time remains exponential.
- **Greedy comparison**: a local density choice does not guarantee optimality for 0–1 knapsack. DP guarantees an optimum for integer capacities but can be costly for very large W.

Theory viva: 20 questions and answers

1. **What is 0–1 knapsack?** A capacity-constrained optimization problem where each item is either taken whole or left out.
2. **Why is it called 0–1?** The decision for each item is binary: 0 for exclude, 1 for include.
3. **Why can the fractional greedy strategy fail?** Whole-item constraints can make a lower-density combination more valuable than the greedy choice.
4. **What is dynamic programming?** A technique that solves overlapping subproblems and stores their results.
5. **What is the DP state?** $DP[i][w]$: maximum value using the first i items within capacity w.
6. **What is the base case for zero items?** Value is zero for every capacity.
7. **What is the base case for zero capacity?** Value is zero because no positive-weight item fits.
8. **What are the two choices for an item?** Exclude it or include it if its weight fits.
9. **What does the max in the transition compare?** The best value from excluding versus including the current item.
10. **What is optimal substructure?** An optimal solution can be built from optimal solutions to smaller subproblems.
11. **What are overlapping subproblems?** The same smaller state is needed in multiple larger calculations.
12. **What is the time complexity of the standard table method?** $O(nW)$.
13. **What is its space complexity?** $O(nW)$ for a full two-dimensional table.
14. **Why is this called pseudo-polynomial?** Complexity depends on the numeric value of W, not only on the number of bits used to represent W.
15. **What does the final table cell mean?** $DP[n][W]$ is the maximum obtainable value using all n items and capacity W.
16. **How can selected items be recovered?** Trace backward through the DP table to see where including an item changed the value.
17. **Can space be optimized?** Yes, a one-dimensional DP array can reduce space to $O(W)$ if capacities are processed in descending order.
18. **Why descending capacity in 1D DP?** It prevents using the same item multiple times in the same iteration.
19. **What is the key difference from unbounded knapsack?** In unbounded knapsack, an item may be selected repeatedly; in 0–1, each item is used at most once.

20. **Give the memory phrase.** "For each item: skip it or take it if it fits."

5 ML Foundations: Common Theory

- **Feature (X)**: input variables used by a model.
- **Target (y)**: value or class the model is asked to predict in supervised learning.
- **Training set**: data used to learn model parameters.
- **Test set**: held-out data used to estimate performance on unseen examples.
- **Preprocessing**: operations such as missing-value handling, encoding, and scaling.
- **Data leakage**: information from test data or future outcomes incorrectly influences training.
- **Overfitting**: strong performance on training data but poor generalization.
- **Underfitting**: model is too simple to capture useful structure.
- **Reproducibility**: fixed random seeds and documented settings help repeat an experiment.
- **Regression**: predicts numeric values; **classification**: predicts discrete labels; **clustering**: groups unlabeled examples.

6 ML Experiment 1: Uber Fare Prediction with Linear Regression and Random Forest

Aim and theory to know

- **Aim**: predict ride fare from ride-related inputs; preprocess data, inspect outliers and correlations, train regression models, and compare metrics.
- **Regression**: predicts a continuous numeric output such as fare.
- **Linear regression**: models a linear relationship between features and target; in simple form $\hat{y} = b_0 + b_1 x$.
- **Coefficients**: describe estimated change in prediction per unit feature change, holding other model inputs fixed.
- **Random forest regression**: combines predictions from many decision trees, typically by averaging.
- **Nonlinearity**: tree ensembles can model interactions and nonlinear patterns without specifying a linear formula.
- **Outliers**: observations unusually far from the general data pattern; they may be errors or genuine rare events.
- **Correlation**: measures association between variables; correlation does not prove causation.
- **MAE**: average absolute prediction error; easier to interpret in target units.
- **MSE / RMSE**: squares errors; RMSE is in target units and penalizes large errors more heavily.
- R^2: compares model error to a baseline based on the target mean; can be negative on test data.
- **Fair evaluation**: compare models on the same held-out test set; fit preprocessing on training data only.

Predict test fares; compare MAE, RMSE and R^2

Deeper theory: data quality and regression foundations

- **Target and predictors**: fare is the target y; pickup/drop-off coordinates, passenger count, and pickup time may be predictors depending on the actual dataset. Identifier columns are not automatically useful predictors.
- **Cleaning**: inspect data types, missing values, duplicates, impossible coordinates, non-positive fares, and invalid passenger counts. Document every removal or imputation rule rather than deleting rows blindly.
- **Outliers**: extreme fares or coordinates can reflect errors or genuine trips. Use domain checks, boxplots/IQR, or robust methods; investigate before removing. Outliers can strongly influence least-squares regression.
- **Correlation**: Pearson correlation measures linear association between two numerical variables, ranges from -1 to +1, and does not prove causation. Low linear correlation does not rule out nonlinear relationships.
- **Linear regression**: predicts $\hat{y} = \beta_0 + \sum_j \beta_j x_j$ and estimates coefficients by minimizing residual sum of squares. Coefficients describe conditional linear effects under the model; correlated predictors can make coefficients unstable.
- **Random forest regression**: fits many decision trees on bootstrap samples, considering random subsets of features at splits, then averages tree predictions. It captures nonlinearities/interactions and is less sensitive to scaling than linear regression.
- **Evaluation**: $R^2 = 1 - \frac{\sum(y-\hat{y})^2}{\sum(y-\bar{y})^2}$; RMSE $= \sqrt{\frac{1}{n}\sum(y-\hat{y})^2}$; MAE $= \frac{1}{n}\sum|y-\hat{y}|$. Lower RMSE/MAE is better; higher R^2 is generally better on the same test set.
- **Leakage and validation**: split before fitting preprocessing transformations; fit imputers/encoders only on training data. Keep the test set untouched until final comparison. Use cross-validation for more reliable model selection.
- **Spatial/time features**: raw latitude/longitude are not a straight-line distance feature; derive haversine distance if appropriate. Pickup time can be expanded into hour/day/season features. Ensure features are available at prediction time.
- **Limitations**: fare can depend on traffic, route, surge pricing, tolls, and policy changes not represented in data. Random forests may extrapolate poorly beyond training ranges; neither model establishes causal effects.

Theory viva: 20 questions and answers

1. **What is the aim of fare prediction?** Estimate a continuous fare value from ride-related features.
2. **Why is this a regression task?** The target is numeric and continuous rather than a class label.
3. **What is linear regression?** A model that predicts a numeric target as a linear combination of input features.
4. **What is the intercept?** The predicted target when model features are zero, under the fitted model.
5. **What is a coefficient?** A learned weight indicating the model's estimated relationship between a feature and prediction.
6. **What is a random forest?** An ensemble of decision trees whose outputs are combined; regression commonly averages tree predictions.
7. **Why can random forests model nonlinear patterns?** Decision trees split feature space into regions and can represent interactions.

8. **What is an outlier?** A value or observation unusually distant from the overall pattern.
9. **Should every outlier be deleted?** No; first determine whether it is a data error or a valid rare event.
10. **What is correlation?** A measure of the strength and direction of association between variables.
11. **Does correlation prove causation?** No.
12. **What does MAE measure?** The mean absolute difference between predicted and actual values.
13. **What does RMSE measure?** The square root of mean squared error; it is in target units and emphasizes large errors.
14. **What does R^2 measure?** Performance relative to predicting the test target mean; a negative value is possible.
15. **Why use a train-test split?** To evaluate generalization on data not used to fit the model.
16. **Why use the same test set for both models?** To make the comparison fair.
17. **Why check pickup and drop-off coordinates?** Impossible or invalid coordinates can distort learned fare patterns.
18. **What is overfitting in this experiment?** A model captures training-specific noise and performs worse on unseen rides.
19. **Does a higher R^2 always guarantee a better model?** Not by itself; consider error metrics, data quality, and intended use.
20. **What is the main evaluation rule?** Compare test-set metrics and explain them in the original fare units where appropriate.

7 ML Experiment 2: Email Spam Detection Using KNN and SVM

Aim and theory to know

- **Aim**: classify emails as spam or not spam and compare KNN with Support Vector Machine (SVM).
- **Binary classification**: predicts one of two classes, such as spam / not spam.
- **KNN**: predicts a class using labels of the K nearest training examples under a distance measure.
- **Distance**: Euclidean distance is common for numeric features; feature representation matters.
- **Scaling**: distance-based methods are sensitive to feature scale.
- **SVM**: seeks a separating boundary with a large margin between classes.
- **Support vectors**: training examples that determine the separating boundary.
- **Kernel**: enables nonlinear decision boundaries by computing similarities in an implicit feature space.
- **Confusion matrix**: records true negatives (TN), false positives (FP), false negatives (FN), and true positives (TP).
- **Precision**: $TP/(TP+FP)$; among predicted positives, fraction truly positive.
- **Recall**: $TP/(TP+FN)$; among actual positives, fraction detected.
- **F1**: harmonic mean of precision and recall; useful when balancing both.
- **Accuracy**: correct predictions divided by all predictions; can mislead with imbalanced classes.



Deeper theory: text representation and classifier geometry

- **Binary classification**: label each email as spam or not spam. The dataset supplies labeled examples; the model learns a decision rule from training data.
- **Text preprocessing**: normalize text consistently, handle missing/duplicate messages, tokenize, and consider lowercasing and punctuation/stop-word handling based on the representation. Avoid removing informative terms blindly.
- **Vectorization**: algorithms require numeric features. Bag-of-words counts term occurrences; TF–IDF weights terms by local frequency and inverse document frequency, reducing the weight of words common across many documents.
- **KNN**: stores labeled vectors and predicts by neighbor voting. Distance is sensitive to feature scale and high-dimensional sparse text; inference can be expensive because many training points may be compared.
- **SVM**: seeks a separating hyperplane with maximum margin. Support vectors are the training points that constrain the boundary. Soft-margin parameter C controls the trade-off between margin width and training errors.
- **Kernels**: linear kernel often suits sparse text; RBF can model nonlinear boundaries but may be more expensive and require tuning. Kernel choice is not a guarantee of better generalization.
- **Imbalanced labels**: accuracy can look high when most emails belong to one class. Inspect spam precision, spam recall, F1, confusion matrix, and possibly PR-AUC.
- **Error trade-off**: false positive = legitimate email marked spam; false negative = spam missed. The preferred balance depends on the application cost of each error.
- **Evaluation protocol**: split messages into train/test sets; fit vectorizer on training text only, transform test text with that fitted vectorizer, tune hyperparameters on training/validation or cross-validation, and evaluate once on test data.
- **Limitations**: vocabulary drift, obfuscated spam, duplicate leakage, and dataset-specific wording can make test performance optimistic compared with real incoming mail.

Theory viva: 20 questions and answers

1. **What is spam detection?** A classification task that labels an email as spam or not spam.
2. **Why is it binary classification?** There are two target classes.
3. **What is KNN?** A non-parametric method that predicts using the labels of nearby training examples.
4. **What does K mean in KNN?** The number of neighbors considered when voting.
5. **What happens when K is too small?** Predictions can be sensitive to noise and individual examples.
6. **What happens when K is too large?** Local patterns may be smoothed out and majority classes may dominate.
7. **Why does scaling matter for KNN?** Features with larger numerical ranges can dominate distance calculations.
8. **What is SVM?** A classifier that seeks a decision boundary with a large margin between classes.
9. **What are support vectors?** Training points that most directly determine the SVM boundary.
10. **What is a kernel?** A function that represents similarity and enables nonlinear decision boundaries.
11. **Give a common SVM kernel.** The radial basis function (RBF) kernel; a linear kernel is another common choice.

12. **What is a confusion matrix?** A table comparing actual and predicted classes.
13. **Define false positive.** An actually negative email predicted as positive, e.g. legitimate mail marked spam if spam is positive.
14. **Define false negative.** An actually positive email predicted as negative, e.g. spam not detected.
15. **What is precision?** $TP/(TP+FP)$.
16. **What is recall?** $TP/(TP+FN)$.
17. **What is F1-score?** $2PR/(P+R)$ for precision P and recall R.
18. **Why can accuracy be misleading?** A model can score highly by predicting the majority class in imbalanced data.
19. **Why compare KNN and SVM?** They use different decision mechanisms and may behave differently on the same features.
20. **What is the key evaluation rule?** Evaluate both models on the same held-out data and consider class-specific errors, not accuracy alone.

8 ML Experiment 3: Bank Customer Churn Prediction Using an ANN

Aim and theory to know

- **Aim**: predict whether a bank customer will leave within the stated future period.
- **Churn**: a customer stops using the service or closes the relationship, according to the dataset definition.
- **ANN**: Artificial Neural Network made of interconnected units arranged in layers.
- **Input layer**: receives feature values; **hidden layers**: learn intermediate representations; **output layer**: produces the prediction.
- **Weights and biases**: learned parameters controlling each unit's computation.
- **Activation function**: introduces nonlinearity; ReLU is common in hidden layers, sigmoid often gives a binary output probability.
- **Forward propagation**: pass inputs through the network to calculate predictions.
- **Loss function**: quantifies prediction error; binary cross-entropy is common for binary classification.
- **Backpropagation**: calculates gradients of loss with respect to parameters using the chain rule.
- **Optimizer**: updates weights to reduce loss; e.g. gradient descent or Adam.
- **Epoch**: one full pass through the training data; **batch size**: number of examples used for an update.
- **Normalization / scaling**: puts numeric features on comparable ranges; fit transformation using training data only.
- **Overfitting controls**: validation data, early stopping, regularization, dropout, and appropriate model size.
- **Evaluation**: confusion matrix and accuracy; precision, recall, and F1 provide class-specific insight.

Predict test probabilities / classes; inspect confusion matrix and metrics

Deeper theory: neural-network learning and churn interpretation

- **Target**: churn label indicates whether a customer leaves within the dataset's defined horizon. It is a binary classification task, not a direct explanation of why an individual will leave.
- **Features**: numeric variables may include age, balance, tenure, credit score; categorical variables such as geography and gender require appropriate encoding. Customer IDs are identifiers and often should not be used as predictive signals.
- **Preprocessing**: inspect missing values and class balance; encode categories; split data; fit scaling/encoding on training data only. Apply exactly the same transformations to test data.
- **Neuron**: computes weighted sum $z = \mathbf{w}^T\mathbf{x} + b$ and applies activation $a = g(z)$. Hidden layers learn intermediate representations.
- **Output layer**: for binary churn, one sigmoid output can estimate a probability in $[0, 1]$; thresholding (often 0.5, but not always optimal) converts probability into a class.
- **Loss and optimization**: binary cross-entropy penalizes probability errors; backpropagation computes gradients using the chain rule; an optimizer such as gradient descent/Adam updates weights to reduce loss.
- **Generalization controls**: validation, early stopping, dropout, L2 regularization, adequate data, and restrained model complexity can reduce overfitting. More layers/neurons do not automatically improve results.
- **Metrics**: accuracy may hide poor churn recall if churners are a minority. Review confusion matrix, precision, recall, F1, ROC-AUC/PR-AUC and threshold-dependent business costs.
- **Class imbalance**: class weights, resampling on training data, or threshold adjustment may help; never resample before the train/test split because that can leak information.
- **Interpretation and fairness**: a high score is not proof of causality or fairness. Check subgroup performance, data quality, privacy, and whether the features are appropriate for business decisions.

Theory viva: 20 questions and answers

1. **What is customer churn?** The event that a customer leaves, according to the dataset's definition.
2. **Why is churn prediction supervised learning?** The historical dataset includes a target label indicating churn status.
3. **What is an ANN?** A network of connected computational units that learns parameters from data.
4. **What are the main layers?** Input, one or more hidden layers, and output.
5. **What do weights do?** They control how strongly input signals contribute to a unit's output.
6. **What is a bias?** A learned offset added to a unit's weighted input.
7. **Why use activation functions?** They introduce nonlinear transformations that let networks model complex patterns.
8. **What is ReLU?** $f(x) = \max(0, x)$, commonly used in hidden layers.
9. **Why use sigmoid for binary output?** It maps a real-valued logit to a value between 0 and 1 that can represent a class probability.
10. **What is forward propagation?** Computing the network output from input through successive layers.
11. **What is a loss function?** A numeric measure of prediction error that training seeks to minimize.
12. **What is binary cross-entropy?** A common loss for binary labels and predicted probabilities.

13. **What is backpropagation?** Computing parameter gradients by applying the chain rule from output loss backward through the network.
14. **What is an optimizer?** The algorithm that uses gradients to update model parameters.
15. **What is an epoch?** One complete pass over the training dataset.
16. **What is batch size?** The number of examples processed for one parameter update.
17. **Why scale input features?** It can improve optimization stability and prevent numeric ranges from dominating.
18. **What is overfitting?** Good training performance but poor performance on unseen data.
19. **Name two ways to reduce overfitting.** Early stopping, regularization, dropout, more data, or a smaller network.
20. **Why use a confusion matrix?** It shows the kinds of correct and incorrect class predictions rather than only one overall score.

9 ML Experiment 4: Diabetes Prediction Using KNN

Aim and theory to know

- **Aim**: classify the dataset's diabetes outcome from medical feature columns using KNN and compute confusion matrix, accuracy, error rate, precision, and recall.
- **Features**: input columns such as glucose, blood pressure, BMI, and other dataset measurements.
- **Target**: the dataset's outcome label; confirm its coding from the supplied data dictionary.
- **KNN classification**: compute distances to training samples, choose K nearest neighbors, and use a majority vote.
- **Euclidean distance**: $d(x, y) = \sqrt{\sum_i (x_i - y_i)^2}$.
- **Scaling**: important because measurements with large scales can dominate distances.
- **Fit scaler on training data only**: transform test data with the same fitted scaler to avoid leakage.
- **Confusion matrix**: for positive class 1, TN = negative correctly predicted, FP = negative predicted positive, FN = positive predicted negative, TP = positive correctly predicted.
- **Accuracy**: $(TP + TN)/(TP + TN + FP + FN)$.
- **Error rate**: $(FP + FN)/(TP + TN + FP + FN) = 1 - \text{accuracy}$.
- **Precision**: $TP/(TP + FP)$; **Recall**: $TP/(TP + FN)$; **F1**: harmonic mean of precision and recall.
- **Medical caution**: a classroom model is not a clinical diagnostic tool; errors and class imbalance matter.



Deeper theory: distance, scaling, and diagnostic metrics

- **Supervised learning**: features X and known labels y are used to learn a classifier; the test set estimates performance on unseen examples.
- **KNN mechanics**: compute distance from a query to training points, select the K smallest distances, then use majority vote or distance-weighted voting. KNN has little explicit training cost but prediction can be expensive.

- **Why scale**: glucose values and BMI may have different numeric ranges. Euclidean distance can be dominated by high-range features unless features are scaled. StandardScaler uses training mean and standard deviation.
- **Choice of K**: small K has low smoothing and high variance; large K smooths more but may underfit. Select K using validation/cross-validation, not by repeatedly checking the test set.
- **Pima diabetes dataset caution**: zero values in some medical columns may encode missing measurements rather than real zero; verify dataset documentation and use a justified cleaning rule. Dataset labels and columns vary by source.
- **Confusion matrix**: with class order [0,1], entries are $[[TN, FP], [FN, TP]]$. Always state which class is positive before interpreting precision or recall.
- **Metric meanings**: accuracy measures total correct; precision asks how many predicted positives are true positives; recall asks how many actual positives were found; F1 is their harmonic mean; error rate is $1 -$ accuracy.
- **Medical-risk interpretation**: false negatives miss positive cases and can be consequential; false positives can cause unnecessary follow-up. Thresholds and metrics should reflect the intended use, not accuracy alone.
- **Limitations**: KNN is sensitive to irrelevant features, scaling, noisy labels, class imbalance, and the curse of dimensionality. It is a teaching model, not a standalone diagnostic tool.

Theory viva: 20 questions and answers

1. **What is the aim of this experiment?** Use KNN to classify diabetes outcome labels and evaluate classification performance.
2. **What type of learning is KNN here?** Supervised classification, because labelled outcomes are provided.
3. **What are X and y?** X contains the input features; y contains the target outcome.
4. **How does KNN classify a new point?** It finds the K nearest training points and votes among their labels.
5. **What does K mean?** The number of neighbors included in the vote.
6. **What is Euclidean distance?** The square root of the sum of squared differences across features.
7. **Why scale medical features?** To prevent features with larger numerical ranges from dominating distance.
8. **Why fit the scaler only on training data?** To avoid test-data information leaking into preprocessing.
9. **What is a confusion matrix?** A table of true/false positives and negatives comparing actual and predicted labels.
10. **What is a true positive?** An actual positive case predicted positive.
11. **What is a true negative?** An actual negative case predicted negative.
12. **What is a false positive?** An actual negative case predicted positive.
13. **What is a false negative?** An actual positive case predicted negative.
14. **State the accuracy formula.** $(TP + TN)/(TP + TN + FP + FN)$.
15. **State the error-rate formula.** $(FP + FN)/(TP + TN + FP + FN)$, equivalently $1 -$ accuracy.
16. **State the precision formula.** $TP/(TP + FP)$.

17. **State the recall formula.** $TP/(TP+FN)$.
18. **What does high recall mean?** A high fraction of actual positive cases are identified.
19. **Why can accuracy be insufficient?** Class imbalance may hide poor performance on the minority class.
20. **Is this model suitable as a standalone diagnosis?** No; it is an educational experiment and not a validated clinical diagnostic system.

10 ML Experiment 5: Sales Customer Segmentation Using K-Means / Hierarchical Clustering

Aim and theory to know

- **Aim**: cluster sales records from `sales_data_sample.csv` and estimate a suitable number of K-Means clusters with the Elbow Method.
- **Unsupervised learning**: discover structure without a labelled target column.
- **Feature selection**: choose meaningful numeric sales/customer variables; conclusions depend on selected columns.
- **Feature scaling**: usually important for distance-based clustering.
- **K-Means**: assigns each point to the nearest centroid, then recomputes centroids as means; repeats until convergence or stopping criteria.
- **Centroid**: mean-based center of a cluster.
- **Inertia**: sum of squared distances between points and their assigned centroids.
- **Elbow Method**: plot inertia against K; look for a bend where further reductions become smaller.
- **No guaranteed exact elbow**: the plot may be ambiguous; interpretability and domain usefulness also matter.
- **Agglomerative hierarchical clustering**: starts with individual points and repeatedly merges clusters.
- **Linkage**: defines inter-cluster distance, e.g. single (minimum), complete (maximum), average, or Ward linkage.
- **Dendrogram**: tree plot that visualizes hierarchical merges; a horizontal cut can define cluster groups.
- **Cluster labels**: arbitrary identifiers, not ranked classes or predictions of a target.



Deeper theory: clustering, distance, and cluster validation

- **Unsupervised learning**: clustering groups observations without a supplied target label. The groups are hypotheses about structure, not guaranteed real customer types.
- **Feature preparation**: sales tables may contain identifiers, dates, product/customer attributes, quantities, prices, and country. Remove IDs from distance calculations unless they have genuine numeric meaning; encode categorical features and choose relevant numeric features deliberately.
- **K-Means objective**: minimize within-cluster sum of squared Euclidean distances (inertia). Alternate assignment to nearest centroid and recomputation of each centroid as the mean of assigned points.

- **Algorithm flow**: choose K and initialize centroids; assign points; recompute means; repeat until assignments/centroids stabilize or an iteration limit is reached.
- **Elbow method**: plot inertia against K. Inertia always tends to fall as K rises; choose a K near the bend where extra clusters give diminishing improvement. The elbow is a heuristic, not proof of the true number of clusters.
- **Initialization and stability**: different starting centroids can lead to different local minima. Use multiple initializations (`n_init`) and a fixed random seed for reproducibility.
- **Scaling**: K-Means relies on distance, so features with larger units can dominate. Standardize/normalize when scales are not intentionally weighted; interpret clusters in original business units afterward.
- **Hierarchical clustering**: agglomerative clustering starts with each point as its own cluster and merges clusters according to linkage. Single, complete, average, and Ward linkage define cluster-to-cluster distance differently.
- **Dendrogram**: visualizes merge sequence and merge distances; a horizontal cut produces a chosen number of clusters. It can help explore structure without selecting K at the start.
- **Comparison and limitations**: K-Means is efficient and centroid-based but sensitive to outliers and works best for roughly compact, spherical clusters; hierarchical clustering provides a hierarchy but can be costly on large datasets and early merges are not undone.
- **Validation**: inspect silhouette score, cluster sizes, stability, and business interpretability in addition to inertia. High silhouette suggests points fit their own cluster better than neighboring clusters.

Theory viva: 20 questions and answers

1. **What is clustering?** Grouping data points so points within a group are similar under the chosen representation and distance.
2. **Is K-Means supervised?** No, it is an unsupervised learning algorithm.
3. **What is the role of K?** It specifies the number of clusters requested from K-Means.
4. **What is a centroid?** The mean-based center of the points assigned to a cluster.
5. **Describe the K-Means loop.** Assign points to nearest centroids, recompute centroids, and repeat until convergence or stopping.
6. **What is inertia?** Within-cluster sum of squared distances to assigned centroids.
7. **What does the Elbow Method estimate?** A reasonable cluster count where extra clusters begin to provide diminishing inertia reduction.
8. **Why does inertia tend to decrease as K increases?** More centroids allow points to be closer to their assigned centers.
9. **Does minimum inertia alone choose the best K?** No; it generally favors more clusters, so the elbow and interpretability matter.
10. **Why scale features before K-Means?** Distance can otherwise be dominated by features with larger numeric scales.
11. **What is hierarchical clustering?** A method that builds nested clusters represented by a hierarchy.
12. **What is agglomerative clustering?** A bottom-up method that begins with single-point clusters and repeatedly merges them.
13. **What is linkage?** The rule for calculating distance between clusters.
14. **What is single linkage?** The minimum pairwise distance between members of two clusters.

15. **What is complete linkage?** The maximum pairwise distance between members of two clusters.
16. **What is a dendrogram?** A tree diagram showing the order and distance of hierarchical cluster merges.
17. **Are cluster IDs meaningful rankings?** No; labels such as 0, 1, and 2 are arbitrary identifiers.
18. **Why does feature choice matter?** The algorithm groups points according to the selected variables, so different features can produce different clusters.
19. **How can clusters be interpreted?** Compare feature distributions or summaries within each cluster and relate them to the business question.
20. **What is a limitation of K-Means?** It requires choosing K, is sensitive to scaling and initialization, and works best for clusters well represented by centroids.

11 Cross-experiment memory map

DAA strategy comparison

Experiment	Core idea	Memory cue
Fibonacci	Recursion vs iteration	Base case; repeated work; $O(n)$ loop
Huffman	Greedy compression	Merge two smallest frequencies
Fractional knapsack	Greedy optimization	Sort by value / weight
0–1 knapsack	Dynamic programming	Skip or take; DP table

ML task and method map

Experiment	Task	Memory cue
Uber fare	Regression	Numeric target; MAE / RMSE / R^2
Spam email	Classification	KNN neighbors; SVM margin; confusion matrix
Bank churn	Classification with ANN	Layers; activation; loss; backpropagation
Diabetes	Classification with KNN	Scale; nearest neighbors; precision / recall
Sales segmentation	Clustering	Centroids; inertia; elbow; linkage

Metrics at a glance

- **Regression**: MAE and RMSE measure error in target units; R^2 compares with a mean-prediction baseline.
- **Classification**: accuracy = correct / total; precision = $TP/(TP+FP)$; recall = $TP/(TP+FN)$; F1 balances precision and recall.
- **Clustering**: inertia summarizes compactness for K-Means; it is not classification accuracy.

Five-step answer structure for theory viva

1. State the definition or aim.
2. Identify the input, output, or decision variable.
3. Explain the core mechanism in 2–3 steps.
4. State the key formula or complexity when relevant.
5. Give one advantage, limitation, or practical caution.